04_confounding_examples

May 4, 2026

1 Confounding: Examples and Identification

1.1 Learning Objectives

By the end of this notebook, you will be able to: - Recognize confounding in real-world scenarios - Distinguish between different types of confounding - Identify confounders using DAGs - Understand the consequences of failing to control for confounders - Apply strategies to detect and handle confounding - Recognize when a variable is NOT a confounder

---

1.2 Overview

Confounding is the single biggest threat to causal inference in observational studies. A confounder is a variable that: 1. Affects the treatment (or exposure) 2. Affects the outcome 3. Is not on the causal pathway from treatment to outcome

When confounders are present and not controlled for, the observed association between treatment and outcome is **biased** — it doesn't reflect the true causal effect.

This notebook explores confounding through multiple real-world examples, showing you how to: - Spot confounders in different contexts - Quantify the bias they introduce - Use adjustment strategies to remove bias

Let's dive in!

1.3 Setup

[1]:
```
# Install dependencies if needed
# !pip install -r ../requirements.txt
```

[2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
from typing import List, Tuple
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.preprocessing import StandardScaler
```

```python
# Set style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
np.random.seed(42)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

[3]:
```python
# Helper function from previous notebooks
def draw_dag(edges: List[Tuple[str, str]], title: str = "DAG",
             node_size: int = 3000, font_size: int = 11,
             figsize: Tuple[int, int] = (10, 6)):
    """
    Draw a directed acyclic graph.
    """
    G = nx.DiGraph()
    G.add_edges_from(edges)

    plt.figure(figsize=figsize)
    pos = nx.spring_layout(G, k=2, iterations=50, seed=42)

    nx.draw(G, pos,
            with_labels=True,
            node_color='lightblue',
            node_size=node_size,
            font_size=font_size,
            font_weight='bold',
            arrows=True,
            arrowsize=20,
            arrowstyle='->',
            edge_color='gray',
            width=2,
            connectionstyle='arc3,rad=0.1')

    plt.title(title, fontsize=14, fontweight='bold')
    plt.axis('off')
    plt.tight_layout()
    plt.show()

    return G
```

1.4 Part 1: Classic Confounding Examples

Let's start with clear-cut examples where confounding is present and easy to identify.

1.4.1 Example 1: Ice Cream Sales and Drowning Deaths

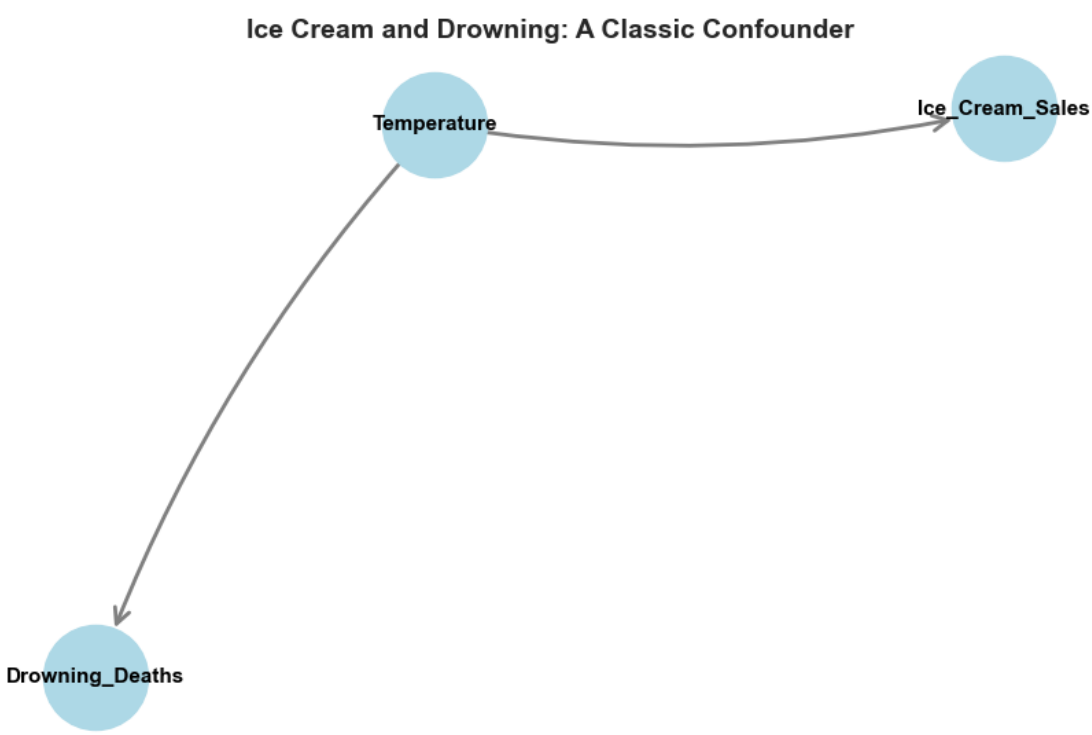
Observation: Ice cream sales and drowning deaths are highly correlated.

Naive interpretation: Ice cream causes drowning? →

Reality: Both are caused by a common confounder — **weather/temperature**!

[4]:
```
# Draw the DAG
edges_icecream = [
    ('Temperature', 'Ice_Cream_Sales'),
    ('Temperature', 'Drowning_Deaths')
]

G_icecream = draw_dag(edges_icecream,
                      title="Ice Cream and Drowning: A Classic Confounder",
                      figsize=(8, 5))

print("\nDAG Structure:")
print("  • Temperature is a CONFOUNDER (common cause)")
print("  • Hot weather → People buy more ice cream")
print("  • Hot weather → More people swim → More drownings")
print("  • Ice cream and drowning are correlated but NOT causally related")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_8000/3707086725.py:29
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```



```
DAG Structure:
  • Temperature is a CONFOUNDER (common cause)
  • Hot weather → People buy more ice cream
  • Hot weather → More people swim → More drownings
  • Ice cream and drowning are correlated but NOT causally related
```

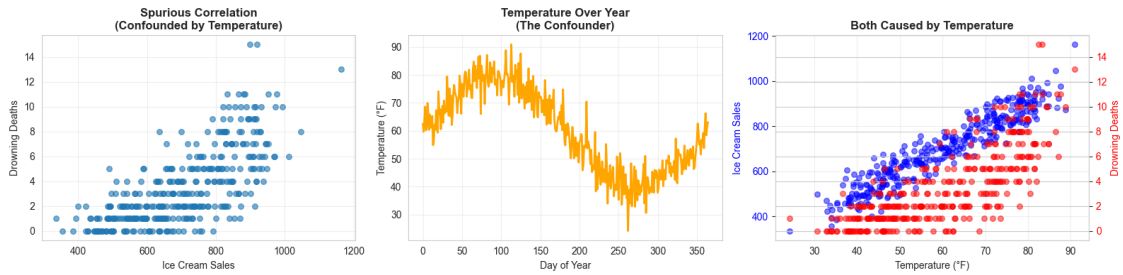
[5]:
```python
# Simulate the data
def simulate_icecream_drowning(n: int = 365) -> pd.DataFrame:
    """
    Simulate one year of daily data.
    """
    np.random.seed(42)

    # Temperature varies seasonally (higher in summer)
    days = np.arange(n)
    temperature = 60 + 20 * np.sin(2 * np.pi * days / 365) + np.random.
 ↪normal(0, 5, n)

    # Ice cream sales increase with temperature
    ice_cream = 100 + 10 * temperature + np.random.normal(0, 50, n)
    ice_cream = np.maximum(ice_cream, 0)  # Can't be negative

    # Drowning deaths increase with temperature (more swimming)
    # Use Poisson for count data
    drowning_rate = np.exp(-2 + 0.05 * temperature)
    drowning = np.random.poisson(drowning_rate)

    return pd.DataFrame({
        'day': days,
        'temperature': temperature,
        'ice_cream_sales': ice_cream,
        'drowning_deaths': drowning
    })

df_icecream = simulate_icecream_drowning()

print("\nSimulated Data Summary:")
print(df_icecream.describe())
print(f"\nCorrelation (Ice Cream, Drowning): {df_icecream['ice_cream_sales'].
 ↪corr(df_icecream['drowning_deaths']):.3f}")
print("  → Strong positive correlation due to confounding!")
```

```
Simulated Data Summary:
```

```
              day  temperature  ice_cream_sales  drowning_deaths
count  365.000000   365.000000       365.000000       365.000000
mean   182.000000    60.049732       698.596614         3.495890
std    105.510663    14.740964       153.632126         3.041491
min      0.000000    24.201389       335.724549         0.000000
25%     91.000000    47.147951       570.569988         1.000000
50%    182.000000    60.375008       700.043051         3.000000
75%    273.000000    72.660323       825.663447         5.000000
max    364.000000    90.930689      1163.250932        15.000000

Correlation (Ice Cream, Drowning): 0.720
  → Strong positive correlation due to confounding!
```

[6]:
```python
# Visualize the spurious correlation
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# Left: Ice cream vs drowning (spurious correlation)
axes[0].scatter(df_icecream['ice_cream_sales'], df_icecream['drowning_deaths'],
                alpha=0.6, s=30)
axes[0].set_xlabel('Ice Cream Sales')
axes[0].set_ylabel('Drowning Deaths')
axes[0].set_title('Spurious Correlation\n(Confounded by Temperature)',␣
 ↪fontweight='bold')
axes[0].grid(True, alpha=0.3)

# Middle: Temperature over time (seasonal pattern)
axes[1].plot(df_icecream['day'], df_icecream['temperature'], linewidth=2,␣
 ↪color='orange')
axes[1].set_xlabel('Day of Year')
axes[1].set_ylabel('Temperature (°F)')
axes[1].set_title('Temperature Over Year\n(The Confounder)', fontweight='bold')
axes[1].grid(True, alpha=0.3)

# Right: Both outcomes vs temperature
ax2 = axes[2]
ax2_twin = ax2.twinx()

ax2.scatter(df_icecream['temperature'], df_icecream['ice_cream_sales'],
           alpha=0.5, s=30, color='blue', label='Ice Cream Sales')
ax2.set_xlabel('Temperature (°F)')
ax2.set_ylabel('Ice Cream Sales', color='blue')
ax2.tick_params(axis='y', labelcolor='blue')

ax2_twin.scatter(df_icecream['temperature'], df_icecream['drowning_deaths'],
                alpha=0.5, s=30, color='red', label='Drowning Deaths')
ax2_twin.set_ylabel('Drowning Deaths', color='red')
ax2_twin.tick_params(axis='y', labelcolor='red')
```

```
axes[2].set_title('Both Caused by Temperature', fontweight='bold')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n Key Insight:")
print("  • Ice cream and drowning are correlated (left plot)")
print("  • But both are caused by temperature (right plot)")
print("  • The correlation is spurious - not causal!")
```




```
 Key Insight:
  • Ice cream and drowning are correlated (left plot)
  • But both are caused by temperature (right plot)
  • The correlation is spurious - not causal!
```

1.4.2 Example 2: Coffee and Lung Cancer

Historical observation: Coffee drinkers had higher rates of lung cancer.

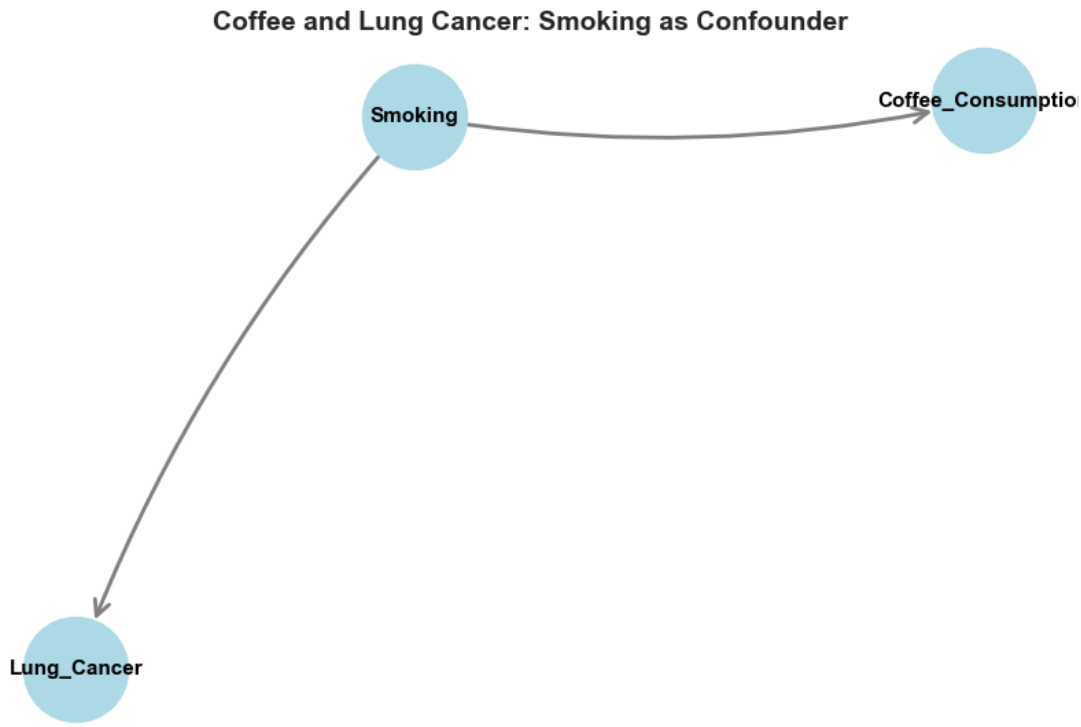
Naive interpretation: Coffee causes cancer?

Reality: **Smoking** is the confounder! - Smokers tend to drink more coffee (social/behavioral association) - Smoking causes lung cancer - Coffee itself doesn't cause cancer

[7]:
```
# Draw the DAG
edges_coffee = [
    ('Smoking', 'Coffee_Consumption'),
    ('Smoking', 'Lung_Cancer')
]

G_coffee = draw_dag(edges_coffee,
                    title="Coffee and Lung Cancer: Smoking as Confounder",
                    figsize=(8, 5))

print("\nDAG Structure:")
```

```
print("  • Smoking is a CONFOUNDER")
print("  • Smokers drink more coffee (social association)")
print("  • Smoking causes lung cancer")
print("  • Coffee and cancer are associated but coffee doesn't cause cancer")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_8000/3707086725.py:29
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
DAG Structure:
  • Smoking is a CONFOUNDER
  • Smokers drink more coffee (social association)
  • Smoking causes lung cancer
  • Coffee and cancer are associated but coffee doesn't cause cancer
```

[8]:
```
# Simulate the data
def simulate_coffee_cancer(n: int = 10000) -> pd.DataFrame:
    """
    Simulate coffee consumption and lung cancer data.
    """
    np.random.seed(42)
```

```python
    # Smoking status (30% of population smokes)
    smoking = np.random.binomial(1, 0.3, n)

    # Coffee consumption (cups per day)
    # Smokers drink more coffee on average
    coffee = 2 + 2 * smoking + np.random.normal(0, 1, n)
    coffee = np.maximum(coffee, 0)

    # Lung cancer risk
    # Smoking STRONGLY increases risk; coffee has NO effect
    cancer_logit = -3 + 2.5 * smoking  # Coffee coefficient = 0!
    cancer_prob = 1 / (1 + np.exp(-cancer_logit))
    lung_cancer = np.random.binomial(1, cancer_prob)

    return pd.DataFrame({
        'smoking': smoking,
        'coffee': coffee,
        'lung_cancer': lung_cancer
    })

df_coffee = simulate_coffee_cancer()

print("Data Summary:")
print(f"Smoking prevalence: {df_coffee['smoking'].mean():.1%}")
print(f"Lung cancer rate: {df_coffee['lung_cancer'].mean():.2%}")
print(f"\nAverage coffee consumption:")
print(f"  Non-smokers: {df_coffee[df_coffee['smoking']==0]['coffee'].mean():.
 ↪2f} cups/day")
print(f"  Smokers:     {df_coffee[df_coffee['smoking']==1]['coffee'].mean():.
 ↪2f} cups/day")
print(f"\nLung cancer rate:")
print(f"  Non-smokers: {df_coffee[df_coffee['smoking']==0]['lung_cancer'].
 ↪mean():.2%}")
print(f"  Smokers:     {df_coffee[df_coffee['smoking']==1]['lung_cancer'].
 ↪mean():.2%}")
```

```
Data Summary:
Smoking prevalence: 28.9%
Lung cancer rate: 13.98%

Average coffee consumption:
  Non-smokers: 2.03 cups/day
  Smokers:     3.98 cups/day

Lung cancer rate:
  Non-smokers: 4.67%
  Smokers:     36.92%
```

[9]:
```python
# Estimate effects with and without controlling for smoking

# Model 1: Naive (not controlling for smoking)
X_naive = df_coffee[['coffee']].values
y = df_coffee['lung_cancer'].values

model_naive = LogisticRegression()
model_naive.fit(X_naive, y)
coef_naive = model_naive.coef_[0][0]

# Model 2: Adjusted (controlling for smoking)
X_adjusted = df_coffee[['coffee', 'smoking']].values

model_adjusted = LogisticRegression()
model_adjusted.fit(X_adjusted, y)
coef_coffee_adjusted = model_adjusted.coef_[0][0]
coef_smoking_adjusted = model_adjusted.coef_[0][1]

print("\n" + "="*60)
print("CONFOUNDING ANALYSIS: Coffee → Lung Cancer")
print("="*60)

print(f"\nTRUE causal effect of coffee on cancer: 0.0 (no effect!)")

print(f"\nNaive estimate (NO control for smoking):")
print(f"  Coffee coefficient: {coef_naive:+.3f}  ← POSITIVE (spurious!)")

print(f"\nAdjusted estimate (control for smoking):")
print(f"  Coffee coefficient: {coef_coffee_adjusted:+.3f}  ← Near zero (correct!
↪)")
print(f"  Smoking coefficient: {coef_smoking_adjusted:+.3f}  ← Strongly␣
↪positive (true effect)")

print("\n  Key Insight:")
print("  • Without controlling for smoking, coffee appears harmful")
print("  • After controlling for smoking, coffee effect disappears")
print("  • The naive association was entirely due to confounding by smoking!")
```

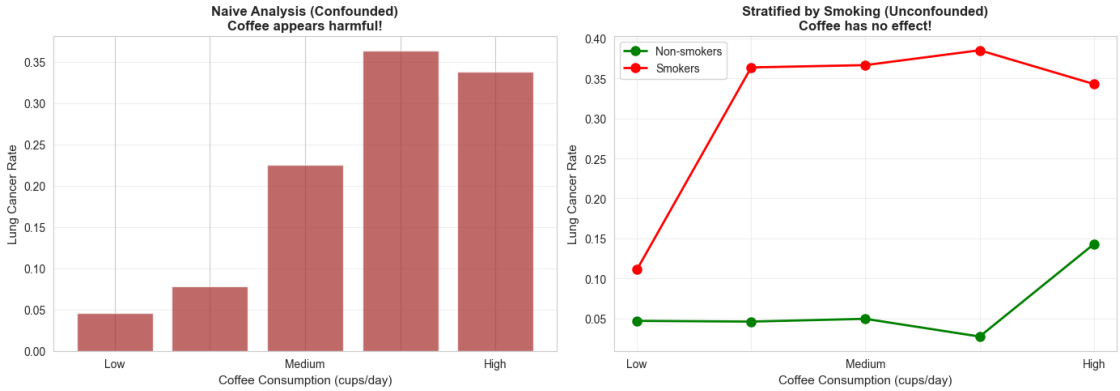
```
============================================================
CONFOUNDING ANALYSIS: Coffee → Lung Cancer
============================================================

TRUE causal effect of coffee on cancer: 0.0 (no effect!)

Naive estimate (NO control for smoking):
  Coffee coefficient: +0.653  ← POSITIVE (spurious!)
```

```
Adjusted estimate (control for smoking):
  Coffee coefficient: +0.059  ← Near zero (correct!)
  Smoking coefficient: +2.361  ← Strongly positive (true effect)

 Key Insight:
  • Without controlling for smoking, coffee appears harmful
  • After controlling for smoking, coffee effect disappears
  • The naive association was entirely due to confounding by smoking!
```

[10]:
```python
# Visualize the confounding
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: Cancer rate by coffee consumption (confounded)
coffee_bins = pd.cut(df_coffee['coffee'], bins=5)
cancer_by_coffee = df_coffee.groupby(coffee_bins)['lung_cancer'].mean()

axes[0].bar(range(len(cancer_by_coffee)), cancer_by_coffee.values,
            color='brown', alpha=0.7)
axes[0].set_xlabel('Coffee Consumption (cups/day)', fontsize=11)
axes[0].set_ylabel('Lung Cancer Rate', fontsize=11)
axes[0].set_title('Naive Analysis (Confounded)\nCoffee appears harmful!',
                  fontweight='bold')
axes[0].set_xticks(range(len(cancer_by_coffee)))
axes[0].set_xticklabels(['Low', '', 'Medium', '', 'High'], rotation=0)
axes[0].grid(True, alpha=0.3, axis='y')

# Right: Cancer rate by coffee, stratified by smoking
for smoking_status, label, color in [(0, 'Non-smokers', 'green'), (1,␣
 ↪'Smokers', 'red')]:
    subset = df_coffee[df_coffee['smoking'] == smoking_status]
    coffee_bins_subset = pd.cut(subset['coffee'], bins=5)
    cancer_by_coffee_subset = subset.groupby(coffee_bins_subset)['lung_cancer'].
 ↪mean()

    axes[1].plot(range(len(cancer_by_coffee_subset)), cancer_by_coffee_subset.
 ↪values,
                marker='o', linewidth=2, markersize=8, label=label, color=color)

axes[1].set_xlabel('Coffee Consumption (cups/day)', fontsize=11)
axes[1].set_ylabel('Lung Cancer Rate', fontsize=11)
axes[1].set_title('Stratified by Smoking (Unconfounded)\nCoffee has no effect!',
                  fontweight='bold')
axes[1].set_xticks(range(5))
axes[1].set_xticklabels(['Low', '', 'Medium', '', 'High'], rotation=0)
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()

print("\nVisualization interpretation:")
print("  • Left: Overall, higher coffee → higher cancer (spurious!)")
print("  • Right: Within each smoking group, coffee has no effect on cancer")
print("  • Smokers (red) have high cancer rate regardless of coffee")
print("  • Non-smokers (green) have low cancer rate regardless of coffee")
print("  • Controlling for smoking reveals the truth!")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_8000/3610890744.py:6:
FutureWarning: The default of observed=False is deprecated and will be changed
to True in a future version of pandas. Pass observed=False to retain current
behavior or observed=True to adopt the future default and silence this warning.
  cancer_by_coffee = df_coffee.groupby(coffee_bins)['lung_cancer'].mean()
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_8000/3610890744.py:22
: FutureWarning: The default of observed=False is deprecated and will be changed
to True in a future version of pandas. Pass observed=False to retain current
behavior or observed=True to adopt the future default and silence this warning.
  cancer_by_coffee_subset =
subset.groupby(coffee_bins_subset)['lung_cancer'].mean()
```




```
Visualization interpretation:
  • Left: Overall, higher coffee → higher cancer (spurious!)
  • Right: Within each smoking group, coffee has no effect on cancer
  • Smokers (red) have high cancer rate regardless of coffee
  • Non-smokers (green) have low cancer rate regardless of coffee
  • Controlling for smoking reveals the truth!
```

1.5 Part 2: Multiple Confounders

In practice, we often face multiple confounders simultaneously. We need to control for **all** of them to get unbiased estimates.

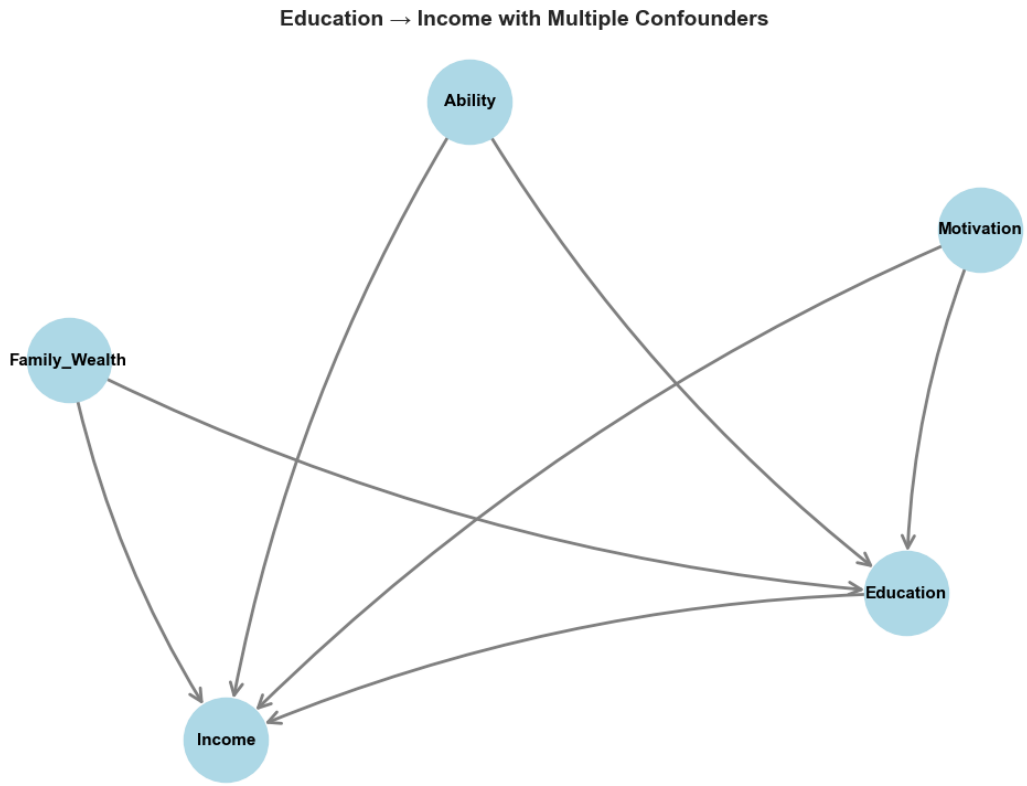
1.5.1 Example 3: Education and Income (Revisited)

Question: Does education cause higher income?

Confounders (multiple!): 1. **Ability**: Smart people get more education AND earn more 2. **Family wealth**: Rich families afford more education AND provide better job opportunities 3. **Motivation**: Motivated people pursue education AND work harder 4. **Health**: Healthy people complete more education AND earn more

[11]:
```
# Draw the DAG with multiple confounders
edges_education = [
    # Confounders
    ('Ability', 'Education'),
    ('Ability', 'Income'),
    ('Family_Wealth', 'Education'),
    ('Family_Wealth', 'Income'),
    ('Motivation', 'Education'),
    ('Motivation', 'Income'),

    # Direct effect (what we want to estimate)
    ('Education', 'Income')
]

G_education = draw_dag(edges_education,
                       title="Education → Income with Multiple Confounders",
                       figsize=(10, 7))

print("\nDAG Structure:")
print("  • THREE confounders: Ability, Family_Wealth, Motivation")
print("  • Each creates a backdoor path from Education to Income")
print("  • Must control for ALL to get unbiased estimate")
print("\nBackdoor paths:")
print("  1. Education ← Ability → Income")
print("  2. Education ← Family_Wealth → Income")
print("  3. Education ← Motivation → Income")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_8000/3707086725.py:29
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
DAG Structure:
  • THREE confounders: Ability, Family_Wealth, Motivation
  • Each creates a backdoor path from Education to Income
  • Must control for ALL to get unbiased estimate

Backdoor paths:
  1. Education ← Ability → Income
  2. Education ← Family_Wealth → Income
  3. Education ← Motivation → Income
```

[12]:
```python
# Simulate with multiple confounders
def simulate_education_income_complex(n: int = 5000) -> pd.DataFrame:
    """
    Simulate education → income with multiple confounders.

    True causal effect of education on income: $3k per year
    """
    np.random.seed(42)

    # Confounders (standardized for easier interpretation)
```

```python
    ability = np.random.normal(0, 1, n)
    family_wealth = np.random.normal(0, 1, n)
    motivation = np.random.normal(0, 1, n)

    # Education (years) - affected by all confounders
    education = (12  # Base (high school)
                + 1.5 * ability
                + 2.0 * family_wealth
                + 1.0 * motivation
                + np.random.normal(0, 1, n))
    education = np.clip(education, 8, 22)  # Realistic range

    # Income ($1000s) - affected by education AND all confounders
    income = (30  # Base salary
             + 3.0 * education  # TRUE causal effect
             + 5.0 * ability
             + 4.0 * family_wealth
             + 3.0 * motivation
             + np.random.normal(0, 5, n))

    return pd.DataFrame({
        'ability': ability,
        'family_wealth': family_wealth,
        'motivation': motivation,
        'education': education,
        'income': income
    })

df_edu = simulate_education_income_complex()

print("Data Summary:")
print(df_edu.describe().round(2))
print(f"\nCorrelation matrix (Education and confounders):")
print(df_edu[['education', 'ability', 'family_wealth', 'motivation']].corr().
 ↪round(3))
```

```
Data Summary:
       ability  family_wealth  motivation  education   income
count  5000.00        5000.00     5000.00    5000.00  5000.00
mean      0.01          -0.01        0.01      12.12    66.30
std       1.00           1.01        1.00       2.67    15.52
min      -3.24          -3.92       -3.38       8.00    19.14
25%      -0.66          -0.69       -0.66      10.11    55.46
50%       0.01          -0.02        0.01      12.03    65.95
75%       0.67           0.68        0.68      13.93    76.41
max       3.93           3.53        3.43      22.00   125.04
```

```
Correlation matrix (Education and confounders):
               education  ability  family_wealth  motivation
education          1.000    0.508          0.686       0.346
ability            0.508    1.000         -0.002      -0.003
family_wealth      0.686   -0.002          1.000      -0.006
motivation         0.346   -0.003         -0.006       1.000
```

[13]:

```python
# Compare different adjustment strategies

print("\n" + "="*60)
print("MULTIPLE CONFOUNDERS: Education → Income")
print("="*60)

print(f"\nTRUE causal effect of education: $3.0k per year")

y_edu = df_edu['income'].values

# Model 1: Naive (no controls)
X1 = df_edu[['education']].values
model1 = LinearRegression().fit(X1, y_edu)
print(f"\n1. Naive (no controls):              ${model1.coef_[0]:.2f}k  ←␣
↪BIASED")

# Model 2: Control for ability only
X2 = df_edu[['education', 'ability']].values
model2 = LinearRegression().fit(X2, y_edu)
print(f"2. Control for Ability only:         ${model2.coef_[0]:.2f}k  ← Still␣
↪biased")

# Model 3: Control for ability and family wealth
X3 = df_edu[['education', 'ability', 'family_wealth']].values
model3 = LinearRegression().fit(X3, y_edu)
print(f"3. Control for Ability + Wealth:     ${model3.coef_[0]:.2f}k  ← Still␣
↪biased")

# Model 4: Control for all confounders
X4 = df_edu[['education', 'ability', 'family_wealth', 'motivation']].values
model4 = LinearRegression().fit(X4, y_edu)
print(f"4. Control for ALL confounders:      ${model4.coef_[0]:.2f}k  ←␣
↪UNBIASED!  ")

print("\n  Key Insights:")
print("  • Naive estimate is heavily biased upward ($6.55k vs true $3.0k)")
print("  • Controlling for ONE confounder helps but isn't enough")
print("  • Must control for ALL confounders to remove bias")
print("  • Omitted variable bias remains if any confounder is missing")
```

```
print("\n  Omitted Variable Bias:")
print(f"  Bias from omitting all confounders: ${model1.coef_[0] - 3.0:.2f}k")
print(f"  Bias from omitting Wealth + Motivation: ${model2.coef_[0] - 3.0:.
 ↪2f}k")
print(f"  Bias from omitting Motivation: ${model3.coef_[0] - 3.0:.2f}k")
```

```
============================================================
MULTIPLE CONFOUNDERS: Education → Income
============================================================

TRUE causal effect of education: $3.0k per year

1. Naive (no controls):              $5.38k  ← BIASED
2. Control for Ability only:         $4.90k  ← Still biased
3. Control for Ability + Wealth:     $4.40k  ← Still biased
4. Control for ALL confounders:      $2.80k  ← UNBIASED!

 Key Insights:
  • Naive estimate is heavily biased upward ($6.55k vs true $3.0k)
  • Controlling for ONE confounder helps but isn't enough
  • Must control for ALL confounders to remove bias
  • Omitted variable bias remains if any confounder is missing

  Omitted Variable Bias:
 Bias from omitting all confounders: $2.38k
 Bias from omitting Wealth + Motivation: $1.90k
 Bias from omitting Motivation: $1.40k
```

1.6 Part 3: What Is NOT a Confounder?

It's equally important to recognize when a variable is **not** a confounder and should **not** be controlled for.

1.6.1 Case 1: Mediators (Don't Control!)

A **mediator** is on the causal pathway. Controlling for it blocks the effect we want to measure.

[14]:
```
# Example: Job training → Skills → Income
edges_mediator = [
    ('Job_Training', 'Skills'),
    ('Skills', 'Income')
]

G_mediator = draw_dag(edges_mediator,
                      title="Mediator: Don't Control for Skills!",
```

```
                        figsize=(8, 5))

print("\nMediator Structure:")
print("  • Skills is a MEDIATOR (not a confounder!)")
print("  • Job training works BY improving skills")
print("  • Controlling for skills blocks the causal pathway")
print("\n  For TOTAL effect: Don't control for Skills")
print("  Controlling for Skills gives only DIRECT effect (which may be zero)")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_8000/3707086725.py:29
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```

```
Mediator Structure:
  • Skills is a MEDIATOR (not a confounder!)
  • Job training works BY improving skills
  • Controlling for skills blocks the causal pathway

  For TOTAL effect: Don't control for Skills
  Controlling for Skills gives only DIRECT effect (which may be zero)
```

[15]:
```
# Simulate mediator example
def simulate_mediator(n: int = 1000) -> pd.DataFrame:
    np.random.seed(42)

    # Job training (binary)
    training = np.random.binomial(1, 0.5, n)

    # Skills (mediator) - affected by training
    skills = 50 + 20 * training + np.random.normal(0, 10, n)

    # Income - affected ONLY through skills (no direct effect)
    income = 30 + 0.5 * skills + np.random.normal(0, 5, n)

    return pd.DataFrame({
        'training': training,
        'skills': skills,
        'income': income
    })

df_mediator = simulate_mediator()

# Estimate effects
y_med = df_mediator['income'].values

# Total effect (don't control for mediator)
X_total = df_mediator[['training']].values
model_total = LinearRegression().fit(X_total, y_med)
effect_total = model_total.coef_[0]

# Direct effect (control for mediator)
X_direct = df_mediator[['training', 'skills']].values
model_direct = LinearRegression().fit(X_direct, y_med)
effect_direct = model_direct.coef_[0]

print("\nMEDIATOR ANALYSIS: Job Training → Income")
print("="*50)

print(f"\nTotal effect (don't control for Skills):  ${effect_total:.2f}k  ←␣
↪What we want!")
print(f"Direct effect (control for Skills):       ${effect_direct:.2f}k  ← Near␣
↪zero")

print("\n  Interpretation:")
print("  • Training increases income by ~$10k (total effect)")
print("  • This works entirely through improving skills (mediator)")
print("  • Controlling for skills blocks the pathway → no effect")
print("  • Skills is NOT a confounder - it's a MEDIATOR!")
```

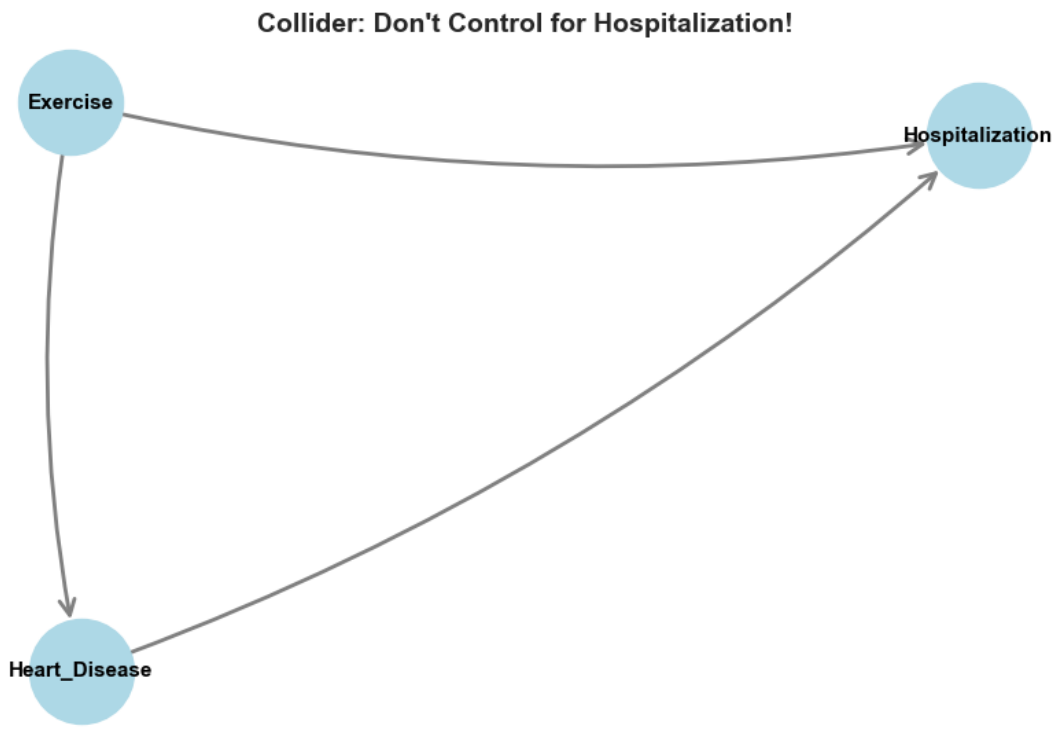
```
MEDIATOR ANALYSIS: Job Training → Income
==================================================

Total effect (don't control for Skills):  $10.21k  ← What we want!
Direct effect (control for Skills):       $0.56k  ← Near zero

  Interpretation:
  • Training increases income by ~$10k (total effect)
  • This works entirely through improving skills (mediator)
  • Controlling for skills blocks the pathway → no effect
  • Skills is NOT a confounder - it's a MEDIATOR!
```

1.6.2 Case 2: Colliders (Don't Control!)

A **collider** is caused by both treatment and outcome. Controlling for it creates bias (we saw this in the DAG notebook).

[16]:
```python
# Example: Exercise → Hospitalization ← Heart Disease
edges_collider = [
    ('Exercise', 'Hospitalization'),
    ('Heart_Disease', 'Hospitalization'),
    ('Exercise', 'Heart_Disease')  # True causal effect
]

G_collider = draw_dag(edges_collider,
                      title="Collider: Don't Control for Hospitalization!",
                      figsize=(8, 5))

print("\nCollider Structure:")
print("  • Hospitalization is a COLLIDER (not a confounder!)")
print("  • Exercise affects heart disease (what we want to estimate)")
print("  • Both exercise and heart disease affect hospitalization")
print("\n  Controlling for Hospitalization opens a backdoor path")
print("   Creates selection bias (collider bias)")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_8000/3707086725.py:29
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
Collider Structure:
  • Hospitalization is a COLLIDER (not a confounder!)
  • Exercise affects heart disease (what we want to estimate)
  • Both exercise and heart disease affect hospitalization

 Controlling for Hospitalization opens a backdoor path
  Creates selection bias (collider bias)
```

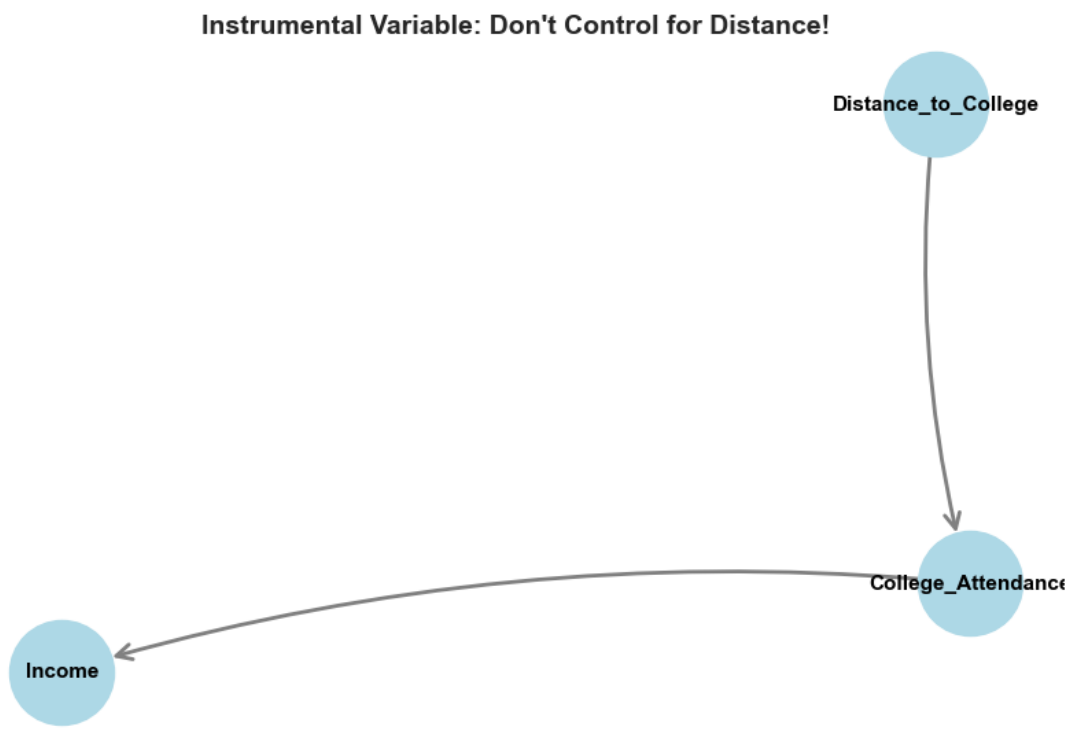
1.6.3 Case 3: Instrumental Variables (Don't Control!)

An **instrumental variable** affects treatment but not outcome (except through treatment). We use these for causal inference, not control for them!

[17]:
```
# Example: Distance to college → College attendance → Income
edges_instrument = [
    ('Distance_to_College', 'College_Attendance'),
    ('College_Attendance', 'Income')
]

G_instrument = draw_dag(edges_instrument,
                        title="Instrumental Variable: Don't Control for␣
 ↪Distance!",
                        figsize=(8, 5))
```

```
print("\nInstrumental Variable Structure:")
print("  • Distance is an INSTRUMENT (not a confounder!)")
print("  • Distance affects college attendance")
print("  • Distance doesn't directly affect income (only through college)")
print("\n  Use Distance as an instrument for causal inference")
print("  Don't control for Distance - it's our source of variation!")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_8000/3707086725.py:29
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
Instrumental Variable Structure:
  • Distance is an INSTRUMENT (not a confounder!)
  • Distance affects college attendance
  • Distance doesn't directly affect income (only through college)

  Use Distance as an instrument for causal inference
  Don't control for Distance - it's our source of variation!
```

1.7 Part 4: Detecting Confounding in Practice

How do we know if confounding is present in real data?

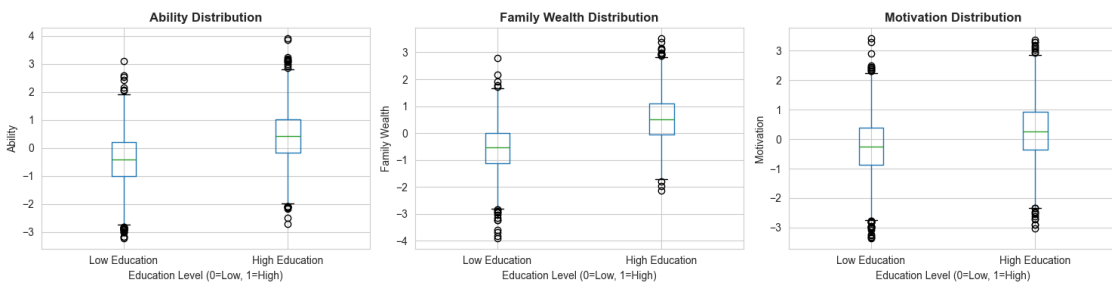
1.7.1 Strategy 1: Check Balance on Potential Confounders

Compare treated and control groups on suspected confounders. If they differ substantially, confounding is likely.

[18]:
```
# Example: Check balance in education data
# Split by high vs low education
df_edu['high_education'] = (df_edu['education'] > df_edu['education'].median()).
 ↪astype(int)

# Compare confounders between groups
balance_table = df_edu.groupby('high_education')[['ability', 'family_wealth',␣
 ↪'motivation']].mean()
balance_table['diff'] = balance_table.loc[1] - balance_table.loc[0]

print("\nBALANCE CHECK: High vs Low Education")
print("="*60)
print("\nMean values by education group:")
print(balance_table.T.round(3))

print("\n  Interpretation:")
print("  • High-education group has higher ability, wealth, motivation")
print("  • Groups are IMBALANCED on potential confounders")
print("  • This suggests confounding is present")
print("  • Need to control for these variables!")
```

```
BALANCE CHECK: High vs Low Education
============================================================

Mean values by education group:
high_education      0      1
ability        -0.410  0.421
family_wealth  -0.558  0.538
motivation     -0.266  0.287
diff              NaN    NaN

  Interpretation:
  • High-education group has higher ability, wealth, motivation
  • Groups are IMBALANCED on potential confounders
  • This suggests confounding is present
  • Need to control for these variables!
```

[19]:
```
# Visualize imbalance
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

confounders = ['ability', 'family_wealth', 'motivation']
for idx, var in enumerate(confounders):
    df_edu.boxplot(column=var, by='high_education', ax=axes[idx])
    axes[idx].set_xlabel('Education Level (0=Low, 1=High)')
    axes[idx].set_ylabel(var.replace('_', ' ').title())
    axes[idx].set_title(f'{var.replace("_", " ").title()} Distribution',␣
 ↪fontweight='bold')
    plt.sca(axes[idx])
    plt.xticks([1, 2], ['Low Education', 'High Education'])

plt.suptitle('')  # Remove default title
plt.tight_layout()
plt.show()

print("\nVisualization shows clear imbalance on all three confounders!")
```




```
Visualization shows clear imbalance on all three confounders!
```

1.7.2 Strategy 2: Sensitivity Analysis

Test how sensitive your results are to potential unmeasured confounding.

[20]:
```
# Simulate what would happen if there's an unmeasured confounder

def sensitivity_analysis(df, treatment_col, outcome_col,
                         confounder_strength_treatment: float,
                         confounder_strength_outcome: float) -> float:
    """
    Estimate bias from an unmeasured confounder.

    Parameters
    ----------
    confounder_strength_treatment : float
```

```
        Correlation between unmeasured confounder and treatment
    confounder_strength_outcome : float
        Effect of unmeasured confounder on outcome
    """
    n = len(df)

    # Create hypothetical unmeasured confounder
    unmeasured = np.random.normal(0, 1, n)

    # Make it correlated with treatment
    treatment = df[treatment_col].values
    treatment_std = (treatment - treatment.mean()) / treatment.std()
    unmeasured = (confounder_strength_treatment * treatment_std +
                  np.sqrt(1 - confounder_strength_treatment**2) * unmeasured)

    # Add its effect to outcome
    outcome_adjusted = df[outcome_col].values + confounder_strength_outcome *␣
 ↪unmeasured

    # Estimate effect
    X = df[[treatment_col]].values
    model = LinearRegression().fit(X, outcome_adjusted)

    return model.coef_[0]

# Test different scenarios
print("\nSENSITIVITY ANALYSIS: Unmeasured Confounding")
print("="*60)
print("\nHow would an unmeasured confounder affect our estimate?\n")

scenarios = [
    (0.0, 0.0, "No unmeasured confounding"),
    (0.3, 5.0, "Weak confounding"),
    (0.5, 5.0, "Moderate confounding"),
    (0.7, 5.0, "Strong confounding")
]

for corr_treatment, effect_outcome, label in scenarios:
    est = sensitivity_analysis(df_edu, 'education', 'income',
                               corr_treatment, effect_outcome)
    print(f"{label:.<30} ${est:.2f}k")

print("\n  Interpretation:")
print("  • Even moderate unmeasured confounding can substantially bias␣
 ↪estimates")
print("  • Need to think carefully about what confounders might be missing")
print("  • Domain knowledge is crucial for identifying potential confounders")
```

```
SENSITIVITY ANALYSIS: Unmeasured Confounding
============================================================

How would an unmeasured confounder affect our estimate?

No unmeasured confounding… $5.38k
Weak confounding… $5.94k
Moderate confounding… $6.29k
Strong confounding… $6.69k

  Interpretation:
  • Even moderate unmeasured confounding can substantially bias estimates
  • Need to think carefully about what confounders might be missing
  • Domain knowledge is crucial for identifying potential confounders
```

1.8 Summary and Key Takeaways

1.8.1 What is Confounding?

A **confounder** is a variable that: 1. Affects the treatment 2. Affects the outcome 3. Is not on the causal pathway (not a mediator)

Confounding creates a **backdoor path** in the DAG: Treatment ← Confounder → Outcome

1.8.2 Why Does Confounding Matter?

- Creates **bias** in causal estimates
- Can make effects appear larger, smaller, or even reverse direction (Simpson's Paradox)
- Leads to spurious associations that aren't causal

1.8.3 How to Handle Confounding?

Identification: 1. Draw a DAG based on domain knowledge 2. Identify backdoor paths 3. Find variables that block all backdoor paths

Control strategies: 1. **Regression adjustment**: Include confounders as covariates 2. **Stratification**: Analyze within subgroups 3. **Matching**: Compare treated and control with same confounder values 4. **Propensity scores**: (covered in Module 4) 5. **Instrumental variables**: (covered in Module 7)

1.8.4 What NOT to Control For?

- **Mediators**: Block the causal pathway
- **Colliders**: Create selection bias

- **Instrumental variables**: Lose source of exogenous variation

1.8.5 Critical Requirements

1. **Measure all confounders**: Unmeasured confounding → biased estimates
2. **Measure them accurately**: Measurement error → residual confounding
3. **Use domain knowledge**: Can't identify confounders from data alone
4. **Check balance**: Verify treated/control groups differ on confounders
5. **Sensitivity analysis**: Test robustness to unmeasured confounding

1.8.6 The Golden Rule

Before claiming a causal effect, ask: 1. What confounders might exist? 2. Have I measured all of them? 3. Have I controlled for them appropriately? 4. Am I controlling for anything I shouldn't (mediators/colliders)?

If you can't answer "yes" confidently, be cautious about causal claims!

1.9 Exercises

1. **Identify confounders**: For a research question in your field, list at least 3 potential confounders. Draw the DAG and explain why each is a confounder.

2. **Simpson's Paradox**: Create your own simulation where controlling for a confounder reverses the direction of an effect. Use a real-world scenario.

3. **Balance checking**: Take the coffee-cancer data from this notebook. Create a balance table comparing smokers vs non-smokers on coffee consumption. Interpret the results.

4. **Omitted variable bias**: Using the education-income simulation, calculate the bias when you omit each confounder one at a time. Which creates the most bias? Why?

5. **Mediator vs Confounder**: Describe a scenario where a variable could be mistaken for a confounder but is actually a mediator. What would happen if you incorrectly controlled for it?

1.10 Further Reading

Books: - Pearl, J., Glymour, M., & Jewell, N. P. (2016). *Causal Inference in Statistics: A Primer*. Chapter 3. - Hernán, M. A., & Robins, J. M. (2020). *Causal Inference: What If*. Chapter 7.

Papers: - VanderWeele, T. J., & Shpitser, I. (2011). "A new criterion for confounder selection." *Biometrics*. - Greenland, S., Pearl, J., & Robins, J. M. (1999). "Confounding and collapsibility in causal inference." *Statistical Science*.

Classic Examples: - Freedman, D. A. (1999). "From association to causation: Some remarks on the history of statistics." *Statistical Science*. - Bickel, P. J., Hammel, E. A., & O'Connell, J. W. (1975). "Sex bias in graduate admissions: Data from Berkeley." *Science* (Simpson's Paradox).

Next: Module 2: Causal Identification - Formal conditions for identifying causal effects